MODUL PRAKTIKUM

PRAKTIKUM DASAR PEMROGRAMAN

Modul 1

Pengenalan Lingkungan Pemrograman dan Struktur Program C



Disusun oleh:

Dr. Desta Sandya Prasvita, S.Komp., M.Kom.

Tim Teaching

PROGRAM STUDI S1 INFORMATIKA

UPN VETERAN JAKARTA

2026

I. Pendahuluan Modul

1. Identitas Modul

Komponen	Keterangan
Mata Kuliah	Praktikum Dasar Pemrograman
Program Studi	S1 Informatika
Semester	1
Modul	1
Judul	Pengenalan Lingkungan Pemrograman dan Struktur Program C
Bahasa Pemrograman	C
Lingkungan Pengembangan	Visual Studio Code + MSYS2 (GCC, GDB)
Alokasi Waktu	1 × 170 menit
Metode	Demonstrasi, latihan terbimbing, eksperimen, dan latihan mandiri
Prasyarat	Tidak ada

2. Indikator Keberhasilan

Mahasiswa dinyatakan berhasil apabila mampu:

1. Mahasiswa mampu mengenali komponen dasar lingkungan pemrograman C.
2. Mahasiswa mampu menjelaskan secara sederhana proses: menulis kode - kompilasi - eksekusi program.
3. Mahasiswa mampu mengenali struktur dasar program C.
4. Mahasiswa mampu menulis, mengompilasi, dan menjalankan program C sederhana.
5. Mahasiswa mampu menggunakan printf() dan komentar sederhana dalam program.

II. Persiapan Praktikum

1. Perangkat Lunak yang Dibutuhkan

Komponen	Fungsi	Status
Visual Studio Code	Editor/IDE utama	Wajib
MSYS2	Lingkungan instalasi toolchain C pada Windows	Wajib
GCC (via MSYS2)	Compiler bahasa C	Wajib
GDB (via MSYS2)	Debugger	Wajib
Microsoft C/C++ Extension (ms-vscode.cpptools)	IntelliSense dan integrasi debugging	Wajib

2. Instalasi dan Konfigurasi Lingkungan

Dilakukan sekali di awal semester.

Langkah 1 — Instal MSYS2

1. Unduh installer MSYS2 dari situs resmi msys2.org.
2. Jalankan installer, gunakan lokasi instalasi default (C:\msys64).
3. Setelah instalasi selesai, buka terminal MSYS2 UCRT64 dari Start Menu.

Langkah 2 — Perbarui paket dan instal toolchain GCC/GDB

Pada terminal MSYS2 UCRT64, jalankan:

```
pacman -Syu
```

Tutup dan buka kembali terminal jika diminta, lalu jalankan:

```
pacman -Su
pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain
```

Toolchain ini sudah mencakup GCC dan GDB.

Catatan: pacman akan menampilkan daftar sub-paket dari grup toolchain (gcc, gdb, binutils, make, dan lainnya) beserta permintaan Enter a selection (default=all). Cukup tekan Enter tanpa mengetik apa pun agar seluruh sub-paket terpasang, lalu tekan Enter/Y sekali lagi saat diminta konfirmasi ukuran unduhan.

Langkah 3 — Tambahkan GCC/GDB ke PATH Windows

1. Buka Edit the system environment variables → Environment Variables.
2. Pada Path (User variables), tambahkan:

```
C:\msys64\ucrt64\bin
```

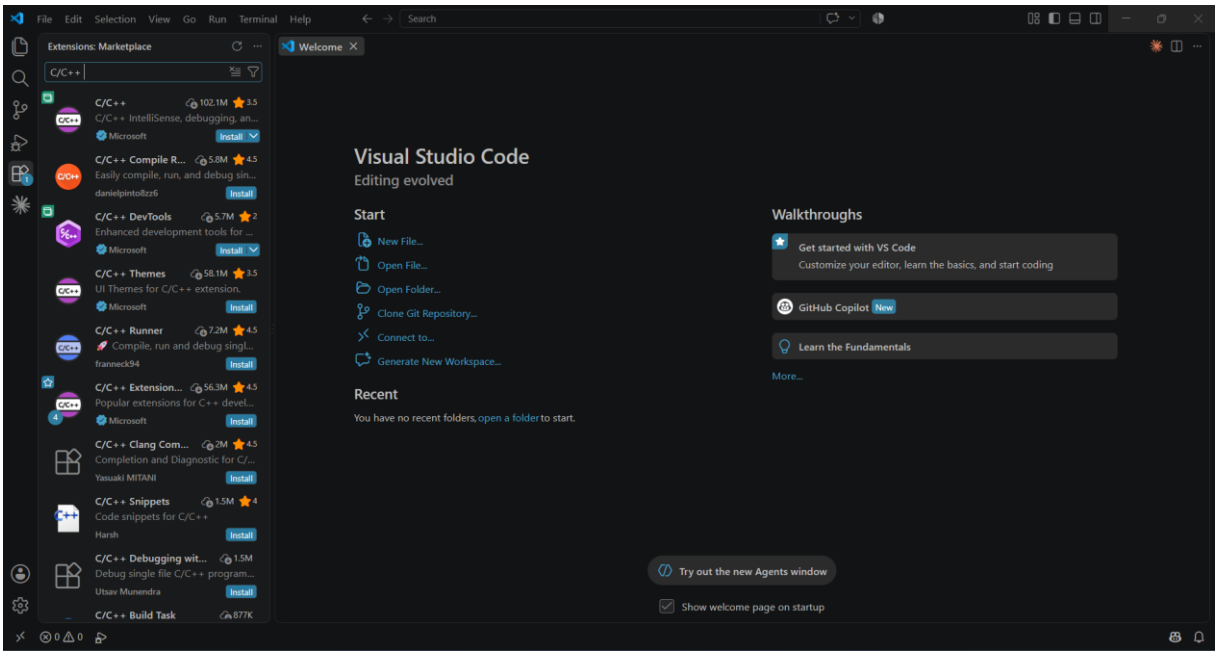
3. Buka Command Prompt/PowerShell baru, lalu verifikasi:

```
gcc --version
gdb --version
```

Jika muncul versi GCC dan GDB, instalasi berhasil.

Langkah 4 — Instal Visual Studio Code

1. Unduh dan instal VS Code dari situs resmi.
2. Buka VS Code, masuk ke tab Extensions.
3. Instal ekstensi C/C++ (dari Microsoft, ms-vscode.cpptools).



Gambar 2. Mencari dan menginstal ekstensi C/C++ (Microsoft) pada tab Extensions VS Code.

Langkah 5 — Uji coba lingkungan

1. Buat folder proyek, misalnya Praktikum_Dasar_Pemrograman.
2. Buka folder tersebut melalui File → Open Folder di VS Code.
3. Buat berkas tes_lingkungan.c berisi program Hello, World!.
4. Buka Terminal → New Terminal di VS Code (pastikan shell terintegrasi terhubung ke PATH yang sudah dikonfigurasi).

Kompilasi:

```
gcc tes_lingkungan.c -o tes_lingkungan.exe
```

Jalankan:

```
.\tes_lingkungan.exe
```

Jika “Hello, World!” muncul di terminal, lingkungan siap digunakan untuk seluruh modul praktikum.

3. Persiapan Folder Praktikum

Sebelum praktikum dimulai, buatlah folder dengan struktur berikut:

```
Praktikum_Dasar_Pemrograman/
└── Modul_01/
    ├── latihan_01.c
    ├── latihan_02.c
    ├── eksperimen.c
    ├── debugging.c
    └── tugas_modul_01.c
```

Gunakan ketentuan penamaan berikut:

- tidak menggunakan spasi;
- tidak menggunakan karakter khusus;
- menggunakan huruf kecil;
- kata dipisahkan menggunakan garis bawah _;
- menggunakan ekstensi .c.

Contoh nama berkas yang benar:

```
program_pertama.c
```

Contoh nama berkas yang tidak disarankan:

```
Program Pertama Saya!.c
```

III. Konsep Dasar

1. Program Komputer

Program komputer adalah kumpulan instruksi yang ditulis untuk membuat komputer melakukan tugas tertentu.

Contoh tugas yang dapat dilakukan program:

- menampilkan informasi;
- menerima data;
- melakukan perhitungan;
- menentukan suatu keputusan;
- mengolah sekumpulan data;
- menyimpan dan membaca data.

Komputer tidak langsung memahami instruksi yang ditulis menggunakan bahasa C. Source code harus diterjemahkan terlebih dahulu menjadi instruksi yang dapat dijalankan oleh komputer.

2. Source Code

Source code adalah instruksi program yang ditulis menggunakan bahasa pemrograman.

Contoh source code bahasa C:

```
#include <stdio.h>


int main(void)
{
    printf("Hello, World!\n");


    return 0;
}
```

Source code bahasa C umumnya disimpan menggunakan ekstensi .c, contoh: hello.c

3. Compiler

Compiler adalah perangkat lunak yang menerjemahkan source code menjadi program yang dapat dijalankan oleh komputer. Pada lingkungan praktikum ini, compiler yang digunakan adalah GCC yang terpasang melalui MSYS2.

Secara sederhana, prosesnya adalah:

```
Source Code  →  Compiler (GCC)  →  Executable Program
```

Contoh: hello.c → proses kompilasi → hello.exe

Nama executable file dapat berbeda bergantung pada sistem operasi dan konfigurasi compiler. Pada Windows, hasil kompilasi umumnya berekstensi .exe.

4. Text Editor dan IDE

Text editor

Text editor digunakan untuk menulis dan mengubah source code.

Integrated Development Environment (IDE)

IDE merupakan lingkungan pengembangan yang biasanya menyediakan:

- editor source code;
- compiler atau integrasi dengan compiler;
- tombol menjalankan program;
- penanda kesalahan;
- terminal;
- debugger;
- pengelolaan folder proyek.

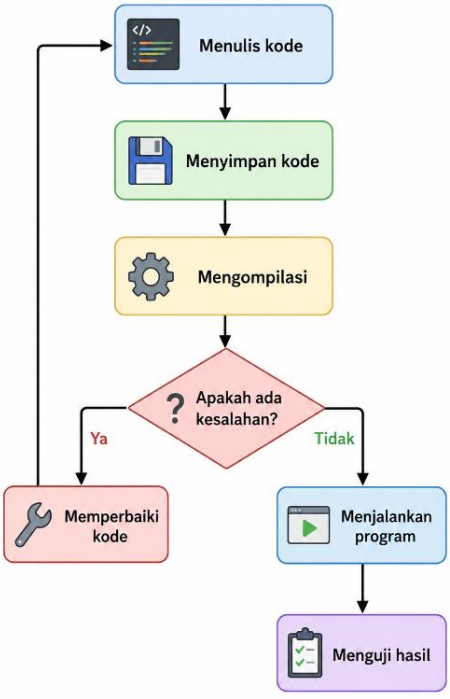
IDE membantu proses pemrograman, tetapi IDE bukan bahasa pemrograman. Pada praktikum ini, Visual Studio Code berperan sebagai editor/IDE, sedangkan GCC (dari MSYS2) berperan sebagai compiler dan GDB sebagai debugger. Bahasa yang digunakan dalam modul ini tetap bahasa C.

5. Tahapan Pembuatan Program

Tahapan umum pembuatan program adalah:

- memahami masalah;
- merancang solusi;
- menulis source code;
- menyimpan source code;
- mengompilasi program;
- memperbaiki kesalahan apabila kompilasi gagal;
- menjalankan program;
- menguji keluaran;
- memperbaiki program apabila hasilnya belum benar;
- mendokumentasikan program.

Alur tersebut dapat digambarkan sebagai berikut:



Gambar 1. Alur tahapan pembuatan program.

6. Struktur Dasar Program C

Perhatikan program berikut.

```
#include <stdio.h>


int main(void)
{
    printf("Selamat datang di Praktikum Dasar Pemrograman.\n");


    return 0;
}
```

Penjelasan setiap bagian:

a. #include

```
#include <stdio.h>
```

Baris tersebut meminta program menyertakan pustaka standar input-output.

Pustaka stdio.h dibutuhkan agar program dapat menggunakan fungsi seperti printf(), dan pada modul berikutnya scanf().

b. Fungsi main()

```
int main(void)
```

Fungsi main() merupakan titik awal eksekusi program C. Ketika program dijalankan, instruksi yang berada di dalam fungsi main() akan diproses.

Kata int menunjukkan bahwa fungsi main() mengembalikan sebuah nilai bilangan bulat.

Kata void pada parameter menunjukkan bahwa fungsi tersebut tidak menerima parameter dari program dalam contoh ini.

c. Kurung kurawal

```
{
}
```

Kurung kurawal menandai awal dan akhir suatu blok program. Pada contoh tersebut, seluruh instruksi utama ditempatkan di antara { dan }.

d. Fungsi printf()

```
printf("Selamat datang di Praktikum Dasar Pemrograman.\n");
```

Fungsi printf() digunakan untuk menampilkan keluaran ke layar. Teks yang akan ditampilkan diletakkan di antara tanda petik ganda " ".

e. Titik koma

```
printf("Belajar bahasa C.\n");
```

Sebagian besar pernyataan dalam bahasa C harus diakhiri dengan titik koma. Apabila titik koma dihilangkan, compiler biasanya akan menghasilkan pesan kesalahan.

f. return 0

```
return 0;
```

Pernyataan tersebut menandakan bahwa fungsi main() selesai dijalankan dengan normal.

7. Komentar dalam Bahasa C

Komentar digunakan untuk memberikan penjelasan pada source code. Komentar tidak dijalankan oleh komputer.

Komentar satu baris

```
// Program pertama menggunakan bahasa C
```

Komentar beberapa baris

```
/*
Program Praktikum Dasar Pemrograman
Modul 1
*/
```

Contoh:

```
#include <stdio.h>

int main(void)
{
    // Menampilkan salam pembuka
    printf("Selamat datang.\n");

    return 0;
}
```

Komentar sebaiknya menjelaskan tujuan atau alasan suatu bagian program, bukan hanya mengulang isi kode.

Kurang informatif:

```
// Menampilkan teks
printf("Selamat datang.\n");
```

Lebih informatif:

```
// Menampilkan pesan pembuka praktikum
printf("Selamat datang.\n");
```

8. Karakter Escape

Karakter escape digunakan untuk mengatur tampilan teks.

Karakter	Fungsi
\n	Pindah ke baris baru
\t	Memberikan tab horizontal
\"	Menampilkan tanda petik ganda
\\	Menampilkan karakter garis miring terbalik
\a	Menghasilkan tanda peringatan pada lingkungan tertentu

Contoh:

```
#include <stdio.h>

int main(void)
{
    printf("Nama\t: Andi\n");
    printf("Program Studi\t: Informatika\n");
    printf("\"Belajar pemrograman dimulai dari memahami dasar.\"\n");

    return 0;
}
```

Perkiraan keluaran:

```
Nama            : Andi
Program Studi   : Informatika
"Belajar pemrograman dimulai dari memahami dasar."
```

IV. Praktikum Terbimbing

1. Praktikum Terbimbing 1 — Membuat Program Pertama

Buat berkas bernama latihan_01.c. Tuliskan program berikut.

```
#include <stdio.h>

int main(void)
{
    printf("Hello, World!\n");

    return 0;
}
```

Langkah pengerjaan

1. Buka folder proyek di Visual Studio Code (File → Open Folder).
2. Buat berkas baru bernama latihan_01.c di dalam folder Modul_01.
3. Ketik program dengan teliti.
4. Simpan berkas (Ctrl+S).
5. Buka Terminal → New Terminal pada VS Code.
6. Lakukan kompilasi melalui terminal.
7. Periksa pesan dari compiler.
8. Jalankan program.
9. Periksa keluarannya.

Kompilasi melalui terminal VS Code

Pastikan terminal yang digunakan adalah Command Prompt atau PowerShell dengan PATH GCC/GDB MSYS2 yang sudah dikonfigurasi (lihat bagian II.2). Perintah kompilasi:

```
gcc latihan_01.c -o latihan_01.exe
```

Jalankan program pada Windows:

```
.\latihan_01.exe
```

Keluaran yang diharapkan:

```
Hello, World!
```

Perintah dan cara penulisan path dapat sedikit berbeda mengikuti jenis terminal yang digunakan (Command Prompt, PowerShell, atau Git Bash), namun proses kompilasi menggunakan GCC tetap sama.

Pertanyaan analisis

1. Apa yang terjadi apabila \n dihapus?
2. Apa yang terjadi apabila tanda petik penutup dihapus?
3. Apa yang terjadi apabila titik koma setelah printf() dihapus?
4. Apakah komentar memengaruhi keluaran program?
5. Dari bagian manakah program mulai dijalankan?

Catat jawaban pada lembar kerja praktikum.

2. Praktikum Terbimbing 2 — Menampilkan Informasi Terstruktur

Buat berkas latihan_02.c. Tuliskan program berikut.

```
#include <stdio.h>
int main(void)
{
    printf("========================================\n");
    printf("       PRAKTIKUM DASAR PEMROGRAMAN      \n");
    printf("========================================\n");
    printf("Nama          : Desta Sandya\n");
    printf("NIM           : 0000000000\n");
    printf("Program Studi : Informatika\n");
    printf("Semester      : 1\n");
    printf("Modul         : Struktur Program C\n");
    printf("========================================\n");
    return 0;
}
```

Ganti nama dan NIM sesuai identitas masing-masing.

Alternatif menggunakan satu printf()

Program yang sama dapat ditulis seperti berikut:

```
#include <stdio.h>
int main(void)
{
    printf(
        "========================================\n"
        "       PRAKTIKUM DASAR PEMROGRAMAN      \n"
        "========================================\n"
        "Nama          : Desta Sandya\n"
        "NIM           : 0000000000\n"
        "Program Studi : Informatika\n"
        "Semester      : 1\n"
        "Modul         : Struktur Program C\n"
        "========================================\n"
    );

    return 0;
}
```

Kedua pendekatan dapat menghasilkan keluaran yang sama. Mahasiswa perlu memahami bahwa source code dapat ditulis dengan lebih dari satu bentuk selama sintaks dan hasilnya benar.

V. Eksperimen dan Tracing

Buat salinan program latihan_02.c dan simpan sebagai eksperimen.c.

Lakukan eksperimen berikut satu per satu. Jangan melakukan semua perubahan secara bersamaan.

No.	Perubahan	Prediksi	Hasil Pengamatan
1	Hapus #include <stdio.h>		
2	Ubah main menjadi Main		
3	Hapus tanda ; pada satu pernyataan		
4	Hapus satu tanda petik ganda		
5	Hapus \n pada beberapa baris		
6	Hapus return 0;		
7	Tambahkan komentar satu baris		
8	Tambahkan karakter \t		

Untuk setiap eksperimen:

- tuliskan prediksi sebelum program dikompilasi;
- kompilasi program;
- catat apakah kompilasi berhasil;
- catat pesan kesalahan atau perubahan keluaran;
- kembalikan program ke kondisi semula sebelum eksperimen berikutnya.

VI. Debugging Challenge

1. Jenis Kesalahan Dasar

Dalam pemrograman, terdapat beberapa jenis kesalahan yang umum terjadi. Pada tahap awal pembelajaran bahasa C, kesalahan dapat dikelompokkan menjadi **kesalahan sintaks, kesalahan logika, dan runtime error**.

a. Kesalahan Sintaks (*Syntax Error*)

Kesalahan sintaks terjadi ketika penulisan program tidak mengikuti aturan bahasa C. Kesalahan ini biasanya terdeteksi oleh *compiler* pada saat proses kompilasi.

Contoh:

```
printf("Selamat datang.\n")
```

Kesalahan pada contoh tersebut adalah tidak adanya titik koma (;) pada akhir pernyataan.

Beberapa contoh kesalahan sintaks antara lain:

- tanda petik tidak ditutup;
- titik koma (;) tidak dituliskan;
- tanda kurung () tidak lengkap;
- kurung kurawal {} tidak lengkap; dan
- penulisan struktur perintah tidak sesuai aturan bahasa C.

Kesalahan sintaks harus diperbaiki terlebih dahulu sebelum program dapat dikompilasi dan dijalankan dengan benar.

b. Kesalahan Logika (*Logical Error*)

Kesalahan logika terjadi ketika program dapat dikompilasi dan dijalankan, tetapi hasil yang ditampilkan tidak sesuai dengan tujuan program.

Pada Modul 1, contoh sederhana kesalahan logika adalah informasi yang ditampilkan tidak sesuai dengan identitas mahasiswa.

```
printf("Semester : 5\n");
```

Program tersebut tetap dapat dijalankan tanpa menghasilkan pesan kesalahan. Namun, informasi yang ditampilkan menjadi salah apabila mahasiswa sebenarnya berada pada semester 1.

Kesalahan logika umumnya tidak terdeteksi oleh *compiler*. Oleh karena itu, programmer perlu memeriksa keluaran program untuk memastikan hasilnya sudah sesuai dengan yang diharapkan.

c. Kesalahan Saat Program Berjalan (*Runtime Error*)

Runtime error adalah kesalahan yang terjadi ketika program sedang dijalankan. Program dapat berhasil dikompilasi, tetapi mengalami masalah ketika melakukan proses tertentu.

Contoh *runtime error* antara lain:

- pembagian dengan nol;
- mengakses lokasi memori yang tidak valid;
- mengakses data di luar batas yang diperbolehkan; dan
- kegagalan ketika membuka atau membaca file.

2. Debugging Challenge

Program berikut memiliki beberapa kesalahan.

```
include <stdio.h>

int Main(void)
{
    printf("Praktikum Dasar Pemrograman\n)
    printf("Modul 1: Struktur Program C\n");

    return 0
}
```

Tugas

1. Salin program tersebut ke berkas debugging.c.
2. Jangan langsung memperbaiki seluruh kesalahan.
3. Kompilasi program terlebih dahulu.
4. Catat pesan kesalahan dari compiler.
5. Identifikasi setiap kesalahan.
6. Perbaiki satu kesalahan.
7. Kompilasi ulang.
8. Ulangi sampai program dapat dijalankan.
9. Catat urutan perbaikan yang dilakukan.

No.	Bagian yang Salah	Jenis Kesalahan	Perbaikan
1			
2			
3			
4			
5			

Keluaran akhir yang diharapkan:

```
Praktikum Dasar Pemrograman
Modul 1: Struktur Program C
```

VII. Latihan Mandiri

1. Level 1 — Melengkapi Program

Lengkapi bagian yang kosong.

```
#include <________>

int ________(void)
{
    printf("Saya sedang belajar bahasa C.____");

    ________ 0;
}
```

Keluaran yang diharapkan:

```
Saya sedang belajar bahasa C.
```

2. Level 2 — Memodifikasi Program

Buat program yang menghasilkan tampilan berikut.

```
+-------------------------------------+
|       PROFIL MAHASISWA INFORMATIKA   |
+-------------------------------------+
| Nama     : [nama mahasiswa]          |
| NIM      : [NIM mahasiswa]           |
| Kelas    : [kelas mahasiswa]         |
| Semester : 1                         |
+-------------------------------------+
```

Ketentuan:

- gunakan minimal lima pernyataan printf();
- gunakan minimal satu komentar;
- gunakan \n;
- source code harus rapi;
- identitas harus sesuai dengan mahasiswa.

3. Level 3 — Membuat Program

Buat program untuk menampilkan jadwal kegiatan perkuliahan sederhana.

Contoh keluaran:

```
========================================
             JADWAL HARI SENIN
========================================
08.00 - 09.40  Algoritma dan Pemrograman
10.00 - 12.50  Praktikum Dasar Pemrograman
13.00 - 14.40  Matematika Diskrit
========================================
```

Ketentuan:

- gunakan judul yang rata dan mudah dibaca;
- gunakan karakter \n dan \t;
- tambahkan komentar berisi identitas pembuat program;
- program harus dapat dikompilasi tanpa kesalahan;
- keluaran tidak boleh terpotong atau tidak beraturan.

4. Challenge — Membuat Ilustrasi Menggunakan Karakter

Buat program yang menampilkan gambar sederhana menggunakan karakter. Contoh:

```
   /\
  /  \
 /____\
 | [] |
 | __ |
 ||  ||
```

Program hanya menggunakan fungsi printf(). Perhatikan penggunaan karakter \\ untuk menampilkan satu garis miring terbalik \ .

5. Poster Ucapan Selamat Datang

Panitia orientasi mahasiswa baru ingin mencetak sebuah poster ucapan selamat datang menggunakan program C. Karena baru mempelajari printf() dan karakter escape, panitia meminta bantuan Anda untuk menuliskan program yang mencetak poster tersebut dengan format yang presisi.

Format Input

Tidak ada input. Program tidak perlu membaca data apa pun dari pengguna.

Format Output

Program mencetak poster persis seperti pada bagian Contoh Output, karakter demi karakter, termasuk jumlah spasi, tanda pemisah, tab, dan baris kosong.

Contoh Output

```
************************************
*      SELAMAT DATANG MAHASISWA      *
*     PROGRAM STUDI INFORMATIKA      *
************************************
Nama Kegiatan:     Orientasi Mahasiswa Baru
Moto:        "Belajar, Berkarya, Berkolaborasi"

Selamat berjuang, mahasiswa baru!
```

Petunjuk Pendekatan

- Program hanya boleh menggunakan #include <stdio.h>, int main(void), pernyataan printf(), dan return 0; — belum menggunakan variabel maupun input.
- Baris pembatas terdiri atas 38 karakter *. Baris judul memiliki susunan * + spasi + teks + spasi + * dengan teks diposisikan di tengah; hitung jumlah spasi kiri dan kanan dengan teliti agar tanda * tetap sejajar dengan baris pembatas.
- Gunakan \t untuk membuat kolom “Nama Kegiatan” dan “Moto” rata, dan perhatikan bahwa baris Moto membutuhkan dua kali \t agar sejajar dengan baris di atasnya.
- Gunakan \" untuk mencetak tanda petik ganda di sekitar moto tanpa mengakhiri string lebih awal.

- Baris kosong di antara poster dan kalimat penutup tetap harus ada — gunakan \n tambahan atau printf("\n"); tersendiri.
- Setelah program selesai ditulis, salin hasil keluaran ke berkas teks lalu bandingkan setiap karakter dengan Contoh Output di atas sebelum dianggap benar.
- Simpan sebagai tantangan_01.c.

VIII. Tugas Praktikum

1. Tugas Praktikum Modul 1 — Kartu Identitas Digital

Buat program C yang menampilkan kartu identitas digital mahasiswa.

Contoh keluaran:

```
================================================
              KARTU IDENTITAS DIGITAL
================================================
Nama            : Desta Sandya Prasvita
NIM             : 123456789
Program Studi   : Informatika
Fakultas        : Ilmu Komputer
Semester        : 1
Mata Kuliah     : Praktikum Dasar Pemrograman
Moto Belajar    : Belajar, berlatih, dan berkembang.
================================================
```

Ketentuan source code

Program wajib:

- menggunakan #include <stdio.h>;
- menggunakan int main(void);
- menggunakan minimal enam pernyataan printf();
- menggunakan \n;
- menggunakan minimal satu karakter \t atau karakter escape lainnya;
- menggunakan komentar yang berisi nama, NIM, dan tujuan program;
- menggunakan indentasi yang konsisten;
- menggunakan return 0;;
- dapat dikompilasi tanpa error menggunakan GCC;
- menghasilkan keluaran yang rapi.

Header komentar

Gunakan format berikut pada awal source code:

```
/*
Nama        :
NIM         :
Kelas       :
Modul       : 1
Deskripsi   : Program menampilkan kartu identitas digital.
*/
```

Berkas yang dikumpulkan

- source code: tugas_modul_01_1_nim.c
- tangkapan layar hasil kompilasi dan eksekusi (terminal VS Code);

- tabel pengujian;
- pernyataan penggunaan AI.

Contoh nama berkas:

```
tugas_modul_01_1_2310511001.c
```

2. Nota Pembelian Alat Tulis (Soal Tambahan)

Selain Kartu Identitas Digital, mahasiswa dapat mengerjakan satu soal tambahan berikut untuk memperoleh poin bonus. Soal ini bersifat opsional, tetapi keluarannya dinilai secara ketat: harus sama persis karakter demi karakter dengan Contoh Output, termasuk spasi dan tab.

Toko alat tulis “Cerdas Stationery” ingin mencetak nota pembelian sederhana. Buat program C yang mencetak nota tersebut persis seperti Contoh Output berikut.

Contoh Output

```
================================
         NOTA PEMBELIAN
================================
Toko : Cerdas Stationery
Kasir: "Rian"
--------------------------------
Pensil 2B    Rp3.000
Buku Tulis   Rp5.000
Penghapus    Rp2.000
--------------------------------
Total:       Rp10.000
================================
Terima kasih atas kunjungan Anda!
```

Ketentuan

- hanya menggunakan #include <stdio.h>, int main(void), pernyataan printf(), dan return 0;;
- gunakan \t untuk menyusun kolom item dan harga agar konsisten dengan Contoh Output;
- gunakan \" untuk mencetak tanda petik ganda di sekitar nama kasir;
- kedua baris pembatas (= dan -) harus memiliki jumlah karakter yang sama seperti pada Contoh Output;
- keluaran diperiksa apa adanya (karakter demi karakter), sehingga kesalahan satu spasi atau tab tetap dianggap tidak sesuai.

Berkas yang dikumpulkan

- source code: tugas_modul_01_2_nim.c
- tangkapan layar hasil kompilasi dan eksekusi (terminal VS Code);
- tabel pengujian;
- pernyataan penggunaan AI.

Contoh nama berkas:

```
tugas_modul_01s_2310511001.c
```

IX. Pernyataan Penggunaan AI

Tambahkan salah satu pernyataan berikut dalam laporan.

Tidak menggunakan AI:

```
Saya menyatakan bahwa tugas ini dikerjakan tanpa menggunakan bantuan
AI generatif.
```

Menggunakan AI:

```
Saya menggunakan [nama perangkat AI] untuk membantu menjelaskan
[bagian yang dibantu]. Saya memeriksa hasilnya dengan cara
[proses pemeriksaan dan pengujian].
```

Mahasiswa tetap bertanggung jawab terhadap kebenaran program yang dikumpulkan.

Daftar Pustaka

Deitel, P., & Deitel, H. (2016). C: How to program (8th ed.). Pearson Education.

Free Software Foundation. (n.d.). GCC, the GNU Compiler Collection: Online documentation. GNU Project. Diakses dari https://gcc.gnu.org/onlinedocs/

Free Software Foundation. (n.d.). GDB: The GNU Project debugger. GNU Project. Diakses dari https://www.gnu.org/software/gdb/

Kernighan, B. W., & Ritchie, D. M. (1988). The C programming language (2nd ed.). Prentice Hall.

Microsoft. (n.d.). C/C++ for Visual Studio Code. Visual Studio Code Documentation. Diakses dari https://code.visualstudio.com/docs/languages/cpp

Microsoft. (n.d.). Configure VS Code for GCC with MinGW-w64 on Windows. Visual Studio Code Documentation. Diakses dari https://code.visualstudio.com/docs/cpp/config-mingw

Microsoft. (n.d.). C/C++ extension for Visual Studio Code (ms-vscode.cpptools) [Perangkat lunak]. Visual Studio Marketplace. Diakses dari https://marketplace.visualstudio.com/items?itemName=ms-vscode.cpptools

MSYS2 Project. (n.d.). MSYS2 — Software distribution and building platform for Windows. Diakses dari https://www.msys2.org/